

Algorithms and Analysis Report

COSC2123/3119

Graph Algorithms in Action: Modelling a Disease Outbreak

Thisul Dilmal — s3988824

Assessment Type	Individual assignment. Submit online via GitHub.
Due Date	Week 11, Thursday May 21, 13:00 (1pm). A late penalty will apply to assessments submitted after 13:00.
Marks	30

1 — Learning Outcomes

This assessment relates to four learning outcomes of the course which are:

- CLO 1: Compare, contrast, and apply the key algorithmic design paradigms: brute force, divide and conquer, decrease and conquer, transform and conquer, greedy, dynamic programming and iterative improvement;
- CLO 2: Compare, contrast, and apply key data structures: trees, lists, stacks, queues, hash tables and graph representations;
- CLO 3: Define, compare, analyse, and solve general algorithmic problem types: sorting, searching, graphs and geometric;
- CLO 4: Theoretically compare and analyse the time complexities of algorithms and data structures; and
- CLO 5: Implement, empirically compare, and apply fundamental algorithms and data structures to real-world problems.

2 — Overview

Across multiple tasks in this assignment, you will model a disease outbreak in the city of Metropolis as a weighted graph to capture residents' contacts, implement and compare algorithms for estimating infection risk through the population, and design a dynamic programming strategy to allocate scarce antiviral treatment as effectively as possible. Some components ask you to critically assess your solutions through both theoretical analysis and controlled empirical experiments, encouraging reflection on the relationship between algorithm design and real-world performance.

I certify that this is all my own original work. If I took any parts from elsewhere, then they were non-essential parts of the assignment, and they are clearly attributed in my submission. I will show I agree to this honor code by typing "Yes": **Yes**

3 — Motivation

The T-Virus has broken out in the city of Metropolis. As the newly appointed Regional Health Director, you have been handed a dossier containing everything the epidemiologists know: a map of every known contact between residents, the probability that each contact transmits the virus in a single day, and the vulnerability of each resident. The clock is ticking — every day the virus spreads further, and your antiviral stockpile is limited.

Your job has four parts:

1. **Build the contact network** (Task A) — represent the city’s contact relationships as a graph, efficiently enough to run algorithms on in real time (implement `graph/adjacency_matrix.py`).
2. **Estimate infection risk** (Task B) — compute the probability that each resident becomes infected over the coming T days, identifying the most at-risk residents before the outbreak grows (implement `transmission/task_b.py` and write report).
3. **Analyse your solution** (Task C) — theoretically and empirically compare your algorithm against the provided baseline across different network structures (write report on analysis of task A and B).
4. **Allocate antiviral treatment** (Task D) — decide who receives the scarce supply of doses, maximising the benefit to the population within the limits of your stockpile (implement `treatment/task_d.py` and write report).

3.1 — Mathematical Background

We model the city of Metropolis as a weighted undirected graph $G = (V, E)$, where:

- each vertex $V_i \in V$ represents a resident, with infection state $S_i \in \{0, 1\}$ where $S_i = 0$ is healthy and $S_i = 1$ is infected;
- each edge $(V_i, V_j) \in E$ represents a contact relationship between two residents; and
- each edge carries a weight $w_{ij} \in (0, 1]$, the probability that an infected resident V_i transmits the virus to a healthy neighbour V_j in a single day.

Each resident V_i also has two personal attributes relevant to the vaccine allocation in Task D:

- a **dosage requirement** $c_i \in \mathbb{Z}^+$, the number of antiviral doses required to vaccinate resident V_i ; and
- a **vulnerability score** $\phi_i \in (0, 1]$, reflecting how severely resident V_i would be harmed if infected.

Over a planning horizon of T days, the probability that an infected resident V_i infects neighbour V_j at least once is:

$$p_{ij} = 1 - (1 - w_{ij})^T$$

Task B: Estimating Infection Risk Over Time (7 marks, 1 page)

Pseudo-code

The Monte Carlo baseline (Algorithm 1) is provided as a style guide; Algorithm 2 is the DP solution following the same style.

Algorithm 1 MonteCarlo(G, s, T, X)

Require: Graph $G = (V, E)$; source s ; horizon T ; simulations X

Ensure: Estimated risk table where $\text{table}[t][i] \approx r_{i,t}$

```
1: for each day  $t = 1$  to  $T$  do
2:   for each simulation  $x = 1$  to  $X$  do
3:     Restart from scratch — only  $s$  is infected
4:     for each day  $d = 1$  to  $t$  do
5:       for each uninfected resident  $V_i$  do
6:         for each infected neighbour  $V_j$  of  $V_i$  do
7:           Infect  $V_i$  with probability  $w_{ij}$ 
8:         end for
9:       end for
10:    end for
11:  end for
12:   $\text{table}[t][i] \leftarrow$  fraction of simulations where  $V_i$  infected
13: end for
14: return table
```

Algorithm 2 InfectionRiskDP(G, s, T)

Require: Graph $G = (V, E)$; source vertex s ; planning horizon T

Ensure: Risk table where $\text{table}[t][i] = r_{i,t}$

```
1: Initialise  $\text{table}[t][i] \leftarrow 0.0$  for all  $t \in [0, T], i \in [0, |V| - 1]$ 
2:  $\text{table}[0][s] \leftarrow 1.0$ 
3: for each day  $t = 1$  to  $T$  do
4:   for each vertex  $V_i \in V$  do
5:     if  $V_i = s$  then
6:        $\text{table}[t][i] \leftarrow 1.0$  ; continue
7:     end if
8:      $\text{still\_healthy} \leftarrow 1 - \text{table}[t-1][i]$ 
9:      $\text{survived} \leftarrow \prod_{V_j \in N(V_i)} (1 - \text{table}[t-1][j] \cdot w_{ij})$ 
10:     $\text{table}[t][i] \leftarrow 1 - \text{still\_healthy} \times \text{survived}$ 
11:   end for
12: end for
13: return table
```

Limitations of the Recurrence

(a) Line 9 of Algorithm 2 multiplies the “fail-to-transmit” probabilities from each neighbour of V_i , which is valid only when those transmission events are independent. This requires that no two neighbours of V_i share a common infection pathway that does not pass through V_i . In

a graph with *cycles*, two neighbours can both trace their infection back to the same ancestor via different routes, making their states correlated. The product treats them as if they were unrelated and produces the wrong probability.

(b) In Figure 3 (the chain $V_0 \xrightarrow{0.8} V_1 \xrightarrow{0.6} V_2$), the recurrence at $t = 3$ for V_1 computes:

$$r_{1,3} = 1 - (1 - r_{1,2}) \cdot (1 - r_{0,2} \cdot 0.8) \cdot (1 - r_{2,2} \cdot 0.6) = 1 - (0.04)(0.2)(1 - 0.480 \times 0.6) \approx 0.994$$

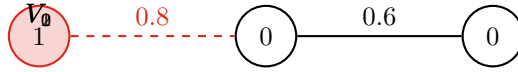


Figure 3: Chain network used in the worked example. V_0 is patient zero.

The term $(1 - r_{2,2} \cdot 0.6) = 0.712$ is the source of error. It treats V_2 as an independent infectious source for V_1 , assigning V_2 a 48% chance of being infectious. But in a chain, V_2 can only be infected through V_1 . In every scenario where V_1 is still healthy at $t = 2$ (the 4% case), V_2 must also be healthy, so its contribution should be zero. The recurrence applies $r_{2,2} = 0.480$ unconditionally, creating a phantom feedback loop $V_0 \rightarrow V_1 \rightarrow V_2 \rightarrow V_1$ that does not exist. The recurrence **over-estimates** V_1 's risk: the correct value is $1 - 0.04 \times 0.2 \times 1.0 = 0.992$, versus the recurrence's 0.994.

(c) The recurrence is exact on **trees** (and forests — acyclic undirected graphs). On a tree there is exactly one path between any pair of vertices, so the neighbours of V_i have completely independent infection histories. The independence assumption holds without approximation and the product over neighbours gives the correct joint escape probability.

Task C: Analysis of Infection Risk Algorithms (8 marks, 2 pages)

Algorithm Complexity Analysis

Let $n = |V|$, $m = |E|$, T = planning horizon, X = simulations.

The Monte Carlo algorithm (Algorithm 1) restarts from scratch for each time step t : it runs X fresh simulations each simulating t days. The total work is $\sum_{t=1}^T X \cdot t \cdot (\text{per-day cost}) = O(T^2 \cdot X \cdot \text{per-day cost})$.

(a) Monte Carlo + adjacency matrix: $O(T^2 \cdot X \cdot n^2)$. Scanning a vertex's row for infected neighbours costs $O(n)$ regardless of degree. Summing over n vertices per day gives $O(n^2)$. Over all time steps: $O(T^2 \cdot X \cdot n^2)$. The dominant cost is the full row scan, which wastes work on zero entries in sparse graphs.

(b) Monte Carlo + adjacency list: $O(T^2 \cdot X \cdot m)$. The neighbour scan visits only actual edges, so the per-day cost is $O(m)$. Total: $O(T^2 \cdot X \cdot m)$. For sparse graphs where $m = O(n)$, this is substantially cheaper than (a).

(c) Dynamic programming + adjacency matrix: $O(T \cdot n^2)$. The DP iterates T days. For each vertex it scans the full matrix row. Total: $O(T \cdot n^2)$. No X factor because the DP is deterministic.

(d) Dynamic programming + adjacency list: $O(T \cdot m)$. Same loop, but neighbour iteration visits only actual edges. Total: $O(T \cdot m)$. For sparse graphs this is the fastest of the four.

The key difference between Monte Carlo and DP is the $T \cdot X$ factor: Monte Carlo repeats earlier work by restarting X simulations per time step, while the DP carries state forward in a single deterministic pass.

Empirical Design

The two most informative variables are n (graph size) and T (planning horizon), since they isolate the representation cost (matrix is $O(n^2)$, list is $O(m)$) and the algorithm cost (DP is $O(T)$, Monte Carlo is $O(T^2)$) respectively. Three experiments were run:

- **Experiment 1:** vary n from 10 to 220, sparse graph ($m = 2n$), $T = 10$, $X = 100$.
- **Experiment 2:** vary n from 10 to 120, dense graph ($m \approx n^2/4$), same T and X .
- **Experiment 3:** vary T from 2 to 20, fixed $n = 50$, sparse graph.

Each data point is the average of three independently seeded runs to reduce noise. T and X are held fixed for the n experiments to isolate graph size effects. X only affects Monte Carlo so holding it constant prevents confounding the representation comparison. Monte Carlo is capped at $n = 70$ in the sparse experiment because it becomes too slow to measure usefully beyond that.

Empirical Analysis

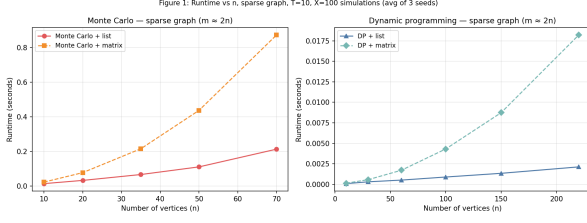


Figure 1: Runtime vs n , sparse graph ($m = 2n$), $T = 10$, $X = 100$, avg 3 seeds.

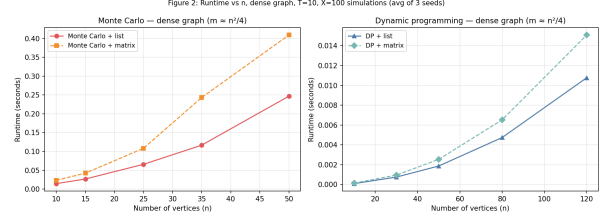


Figure 2: Runtime vs n , dense graph ($m \approx n^2/4$), same parameters.

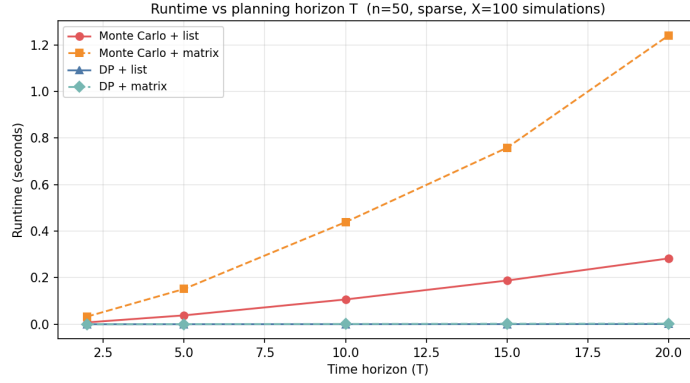


Figure 3: Runtime vs planning horizon T , $n = 50$, sparse graph.

Figure 1 (sparse, varying n): At $n = 70$, MC+matrix reaches 0.87 s while DP+list is under 0.002 s — a roughly 450 $\times$ difference. Within Monte Carlo, the matrix is about 4 $\times$ slower than the list, matching the $O(n^2)$ vs $O(n)$ prediction for sparse graphs.

Figure 2 (dense): At $n = 50$ dense, MC+list (0.247 s) and MC+matrix (0.409 s) are only 1.7 $\times$ apart, compared to 4 $\times$ in the sparse case. When $m \approx n^2/4$, the list traverses nearly as many entries as the matrix scans, so the representations converge.

Figure 3 (varying T): From $T = 5$ to $T = 20$, MC+list grows from 0.038 s to 0.283 s (about 7.5 $\times$), broadly consistent with quadratic growth. DP stays essentially flat (0.0002 s to 0.001 s), consistent with $O(T \cdot m)$ where m is fixed and small.

Reflection

The empirical results match the theoretical predictions in direction. For Metropolis, DP with an adjacency list is clearly the right choice. A city contact graph is sparse (each resident has a bounded number of direct contacts), so $m = O(n)$ and the DP runs in $O(T \cdot n)$ time. MC+matrix is around 450 $\times$ slower at moderate sizes.

If transmission rates or contacts updated daily, the solver would need to rerun with a fresh graph each day. DP is still faster: one pass at $T = 1$ costs $O(m)$, while a useful Monte Carlo run requires many simulations regardless of T . The adjacency list advantage holds in either case.

Task D: Antiviral Allocation (10 marks, 2 pages)

Algorithm Design

This is the 0/1 knapsack problem. Each resident V_i has an integer cost c_i (dosage requirement) and a real-valued benefit $b_i = r_{i,T}$ (infection risk from Task B). The goal is to select a subset of healthy residents maximising total benefit without exceeding the dose budget C , with ties broken in favour of the selection using fewer doses.

A top-down memoised table $memo[i][c]$ is used, where each cell stores the pair $(max_benefit, min_doses)$ achievable using the first i residents with c doses remaining.

Base cases: $memo[0][c] = (0.0, 0)$ for all c , and $memo[i][0] = (0.0, 0)$ for all i (no residents or no capacity — benefit is zero).

Recurrence: for person i with cost c_i and benefit b_i :

$$memo[i][c] = \begin{cases} memo[i-1][c] & \text{if } c_i > c \\ \arg \max_{\text{benefit}} \{ memo[i-1][c], memo[i-1][c - c_i] + (b_i, c_i) \} & \text{otherwise} \end{cases}$$

On a benefit tie, the option using fewer doses is chosen.

Backtracking: starting from $memo[n][C]$, at each step (i, c) , if $memo[i][c]$ differs from $memo[i-1][c]$ in benefit or doses, person i was included. Subtract c_i from c and move to $(i-1, c - c_i)$. Otherwise person i was skipped.

Complexity Analysis

The memo table has $(n+1)$ rows and $(C+1)$ columns. Each cell is computed at most once in $O(1)$ time. **Time complexity:** $O(n \cdot C)$. **Space complexity:** $O(n \cdot C)$ for the table. n is the number of eligible residents and C is the total dose budget. The C factor makes this pseudo-polynomial: runtime grows with the numeric value of C rather than its bit-length.

Extension 1: Triage

To guarantee that more medically vulnerable residents receive doses before less vulnerable ones, residents are partitioned into K tiers ordered by vulnerability score (highest first). The knapsack DP is run on each tier sequentially, passing the remaining budget forward.

Modified algorithm: for each tier k from 1 to K , run $KNAPSACK(tier_k, C_{remaining})$ and subtract the doses used before processing tier $k+1$.

Complexity: $O(K \cdot n_{\max} \cdot C)$ where $n_{\max} = \max_k |tier_k|$. In the worst case (one tier) this reduces to $O(n \cdot C)$.

Counterexample (why ignoring vulnerability fails), with $C = 5$:

Resident	Vulnerability	Benefit	Cost
V_1	0.9 (high)	0.50	5
V_2	0.1 (low)	0.35	3
V_3	0.1 (low)	0.25	2

Without triage, the DP selects $V_2 + V_3$ (total benefit 0.60, cost 5). V_1 — the most medically vulnerable resident — receives nothing because the DP only sees aggregate benefit, not who stands to be harmed most. With triage, V_1 is allocated doses first as the highest-vulnerability tier.

Extension 2: Interdependent Vaccinations

The current DP relies on each resident’s benefit $b_i = r_{i,T}$ being a fixed constant independent of who else was vaccinated. This is what allows the problem to decompose: when deciding whether to include person i , the value of that decision does not depend on anything else in the selection.

In reality, vaccinating V_i removes them from the network, which reduces the infection risk for their neighbours. So b_j is not fixed — it changes depending on the vaccination set. The subproblem structure breaks down.

Concrete example. Consider the chain $V_0 \rightarrow V_1 \rightarrow V_2 \rightarrow V_3$ with budget $C = 3$, costs $c_1 = 3$, $c_2 = 2$, $c_3 = 1$, and independently computed risks $r_1 = 0.8$, $r_2 = 0.7$, $r_3 = 0.6$. The DP selects $V_2 + V_3$ (total benefit 1.3, cost 3). But vaccinating V_1 alone (cost 3) would block all downstream transmission, dropping the true risks of V_2 and V_3 to near zero. The DP cannot see this because it treats the pre-computed risks as fixed.

Why it is significantly harder. To find the truly optimal vaccination set under interdependence, we would need to re-run the Task B solver for every candidate subset, since each subset produces a different effective contact network. That is 2^n subsets, each requiring $O(T \cdot m)$ computation, for a total of $O(2^n \cdot T \cdot m)$. Without the independence assumption there is no known polynomial-time formulation, and the search space grows exponentially with the number of residents.